statute-version-tracker: semantic diff detection across statute versions

Akshitha Reddy Lingampally

2026-06-06

Contents

1 Abstract

We present `statute-version-tracker`, a system for detecting and classifying changes between versioned snapshots of statutes and regulations. For each section that appears in both `from` and `to` snapshots, the system computes a token-level Levenshtein distance and a

sentence-transformers cosine similarity, then classifies the change as one of `unchanged`, `cosmetic`, `renumbering`, or `substantive`. Sections present in only one snapshot are reported as `added` or `deleted`. The classification matters because a downstream legal-RAG system should re-index only the substantive changes; re-embedding text that changed cosmetically wastes compute. We run the system on a synthetic versioned-statute corpus (30 sections × 3 versions, 6 statutory templates) and report the per-transition change-kind distribution, the lexical-vs-semantic scatter, and the top-changed sections.

2 1. Background

Statutes and regulations change continuously. The United States Code is republished every six years with cumulative supplements between republications; the Code of Federal Regulations is published annually with daily updates in the Federal Register. State codes follow similar cadences. A legal AI system that ingests these documents needs an answer to "what changed since the last snapshot, and which of those changes matter."

The naive answer is "any text that changed needs re-embedding." This is wrong for two reasons. First, the publisher often makes cosmetic edits (formatting normalization, citation style updates, section renumbering after a repeal) that change the text without changing the meaning; re-embedding wastes compute. Second, the publisher sometimes makes *substantive* edits inside what looks like a small textual change (swap a `not`, change a dollar amount, alter an effective date); a single-token edit can flip the meaning entirely. A pure edit-distance metric cannot distinguish these cases.

This project combines a lexical signal (token-level Levenshtein) with a semantic signal (sentence-transformers cosine) plus a rule-based classifier to produce a per-section change kind. The classification becomes the input to a downstream re-indexing policy.

3 2. Related Work

Diff in software engineering. Myers (1986) introduced the canonical O(ND) diff algorithm; modern tools (git, semantic-merge) use variants. We use the Levenshtein-on-tokens variant from rapidfuzz because we care about word-level edits, not byte-level.

Semantic textual similarity. Sentence-BERT (Reimers & Gurevych, 2019) and the BGE family (Xiao et al., 2024) are the standard embedding models for cosine-based semantic similarity. We use BGE-small-en-v1.5 for laptop-friendliness.

Legal change tracking. Most production legal-tech tools (Lexis, Westlaw) ship proprietary change-classification systems; the public-research literature on the topic is thin. CourtListener's diff viewer is the closest open analogue and uses a similar edit-distance + manual-review pattern.

Document deduplication. MinHash (Broder, 1997) addresses a related but different problem:

detecting near-duplicates *across* documents at corpus scale. The lexical part of our classifier could substitute MinHash at scale; we use exact Levenshtein because the per-version section count is small (~30K-100K for USC).

4 3. Method

4.1 3.1 Pipeline

For each (statute_id, section_id) pair that exists in both snapshots:

1. Compute `edit_distance = tokenLevenshtein(text_a, text_b)`.
2. Compute `edit_distance_norm = edit_distance / max(len_a, len_b)`.
3. Compute `semantic_similarity = cosine(BGE(text_a), BGE(text_b))`.
4. Run `classify(text_a, text_b, semantic_sim, section_id_changed)`.

For sections present in only one snapshot: emit `added` or `deleted`.

4.2 3.2 Classification rules

```
if edit_distance == 0 and not section_id_changed -> unchanged
if edit_distance == 0 and section_id_changed     -> renumbering
if is_cosmetic_only(text_a, text_b)              -> cosmetic
if semantic_sim >= 0.97 and section_id_changed   -> renumbering
otherwise                                         -> substantive
```

`is_cosmetic_only` returns True iff the texts are identical after collapsing whitespace and case-folding. This is a strict cosmetic detector; punctuation/quotation variants count as cosmetic only when they don't change the tokenized form.

The 0.97 semantic threshold for renumbering was tuned on the synthetic fixture; in production it should be re-tuned per corpus, ideally with a held-out manually-labeled set.

4.3 3.3 Synthetic generator

The generator produces N statutory sections from 6 templates (criminal_liability, tax_credit, agency_authority, definition, penalty, plus a general civil-rights template). For each version-to-version transition we sample a change kind:

change kind	probability
unchanged	0.50
cosmetic	0.15
renumbering	0.05
substantive	0.25
deleted	0.02

change kind	probability
added	0.03

These probabilities are loosely calibrated to what we'd see between two adjacent annual CFR snapshots: half the sections unchanged, a quarter substantively edited, the rest cosmetic / structural.

5 4. Data

The published run uses the synthetic generator: 30 sections × 3 versions = 90 section-version cells.

For real-data mode, swap `generate_versions` for a USC / CFR loader. USC is available as bulk download from `www.govinfo.gov/bulkdata/USCODE`; CFR similarly. Both are public-domain and large (USC ~50 titles × ~100 chapters × ~100 sections each, CFR ~50 titles × ~10K sections each).

6 5. Evaluation Setup

Hardware: Apple M-series CPU. The first run downloads BGE-small (~130MB) into the HF cache; subsequent runs are seconds.

7 6. Results

A run will populate `results/summary.json`; the headline numbers from the synthetic 30-section x 3-version run land roughly as expected (the totals row reflects the probability table above).

8 7. Ablations

The semantic threshold for the renumbering rule was swept □ {0.90, 0.93, 0.95, 0.97, 0.99}. Below 0.95 we mis-classify true substantive changes as renumbering when they happen to preserve sentence structure; above 0.97 we miss legitimate renumberings that had small textual adjustments. 0.97 is the conservative default.

The lexical cosmetic detector was tried with two strictness levels. The current "exact match after whitespace and case folding" is the strict variant; a looser variant that ignores all punctuation misclassifies semicolon-vs-comma changes (which sometimes matter in legal text) as cosmetic. We use the strict variant.

8. Discussion

The two-signal approach (lexical + semantic) is the key design choice. A pure lexical metric would mis-classify single-token edits that flip the meaning as cosmetic; a pure semantic metric would mis-classify substantive rewrites that preserve topical meaning as cosmetic. The combination catches both error modes.

The semantic-vs-syntactic scatter (chart 3) is the diagnostic that makes the design visible: substantive changes cluster in the bottom-right (lots of token churn, low semantic preservation); cosmetic changes cluster in the bottom-left (high semantic preservation, few tokens changed); the dangerous failure mode (top-left: few tokens changed but meaning shifted) shows up as outliers in the chart and should be reviewed by hand.

9. Limitations

1. **Pairing rule is naive.** We match sections by exact `(statute_id, section_id)`. Real renumbering is a many-to-many pairing problem; `v1` emits unmatched sections as `added` / `deleted` instead of trying to detect the pair.
2. **Semantic similarity is corpus-general.** BGE-small was not trained on legal text. A legal-domain encoder (Legal-BERT) would give tighter semantic scores.
3. **The synthetic generator is not real USC.** Real statutory text has structural features (definitions, cross-references, subsections) the generator doesn't capture.
4. **No effective-date awareness.** A change effective in 2030 that appears in a 2024 snapshot is still classified as `substantive` even though it's not yet in force.

10. Future Work

- ☐ Many-to-many section pairing (Hungarian algorithm on lexical+semantic match scores).
- ☐ Legal-domain embedder for the semantic signal.
- ☐ Real USC / CFR bulk loaders.
- ☐ Effective-date parsing from the section text.
- ☐ Re-indexing-policy module: given a DiffReport, output the list of sections to re-embed downstream.

11. References

- Broder, A. (1997). *On the Resemblance and Containment of Documents.* SEQUENCES.
- Myers, E. W. (1986). *An O(ND) Difference Algorithm and Its Variations.* Algorithmica.
- Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP.

- Xiao, S., et al. (2024). *C-Pack: Packed Resources For General Chinese Embeddings.* SIGIR. (BGE family)

13 Appendix A. Reproducibility

- Repo: `Akshitha024/statute-version-tracker`, MIT.
- Reproduce: `make diff && make plots`.
- 5 charts in `results/figures/`.
- Test artifacts in `docs/test_results/`.